

Reinforcement Learning: Training an Agent in the Gymnasium

Adam Maltoni and Ibón de Mingo Arroyo

May 2025

Abstract

This lab assignment explores reinforcement learning by training an agent in the Gymnasium environment. We implement and compare Q-learning and SARSA algorithms across various environments, analyze the effects of exploration parameters, and introduce Deep Q-Learning (DQN) for more complex state spaces.

Contents

1	Exercise 1: The gymnasium	2
2	Exercise 2: Q-learning	2
2.1	Step 0: Set up and understand Frozen Lake environment	2
2.2	Step 1: Greedy and Epsilon greedy policies	4
2.3	Step 2: Train the RL agent	4
2.4	A more challenging problem	5
2.5	Slippery environment	6
3	Exercise 3: Train the RL agent using SARSA	7
4	Exercise 4: Compare SARSA and Q-learning	8
4.1	What is episode reward in RL and how is it related to the performance of an RL agent?	8
4.2	What is episode length in RL and how is it related to the performance of an RL agent?	8
4.3	What is training error in RL and how is it related to the performance of an RL agent?	8
5	Exercise 5: Compare SARSA and Q-learning plots	9
6	Exercise 6: Train a deep Q-learning agent	9
6.1	Deep Q-Learning Agent	10
7	References	11

1. Exercise 1: The gymnasium

Training a reinforcement learning agent at the gymnasium.

- Basic usage: https://gymnasium.farama.org/introduction/basic_usage/
- Training an RL-agent: https://gymnasium.farama.org/introduction/train_agent/

```

1 %load_ext autoreload
2 %autoreload 2
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import gymnasium as gym
6 import imageio
7 import os
8 from tqdm.notebook import tqdm
9 import rl_utils
10 import time
11 from IPython.display import display, clear_output
12 from reinforcement_learning import (
13     sarsa_learning,
14     q_learning,
15     greedy_policy,
16     epsilon_greedy_policy,
17 )

```

```

1 import gymnasium as gym
2 env = gym.make('CartPole-v1')

```

```

1 env = gym.make("LunarLander-v3", render_mode="human")
2 observation, info = env.reset()
3 episode_over = False
4 while not episode_over:
5     action = env.action_space.sample() # agent policy that uses the
        observation and info
6     observation, reward, terminated, truncated, info =
        env.step(action)
7     episode_over = terminated or truncated
8 env.close()

```

2. Exercise 2: Q-learning

We're now ready to code our Q-Learning algorithm.

2.1. Step 0: Set up and understand Frozen Lake environment

Let's begin with a simple 4x4 map and non-slippery, meaning the agent always moves in the desired direction. We add a parameter called `render_mode` that specifies how the environment should be visualised. In our case because we want to record a video of the environment at the end, we need to set `render_mode` to `rgb_array`.

As explained in the documentation `rgb_array`: Return a single frame representing the current state of the environment. A frame is a `np.ndarray` with shape (H, W, 3) representing RGB values for an H (height) times W (width) pixel image.

```

1 small_environment = gym.make(
2     'FrozenLake-v1',
3     map_name='4x4',
4     is_slippery=False,
5     render_mode='rgb_array',
6 )

```

Let's see what the environment looks like:

```

1 state, info = small_environment.reset() # observation state
2 action_names = {0: 'Left', 1: 'Down', 2: 'Right', 3: 'Up'}
3 print(state)
4 print(info, '(Probability that the action has led to the current
5     state)')
6 n_states = small_environment.observation_space.n
7 print("There are ", n_states, " possible states")

```

```

1 n_actions = small_environment.action_space.n
2 print("There are ", n_actions, " possible actions")
3 print(action_names)
4 fig, ax = plt.subplots()
5 game_image = ax.imshow(small_environment.render())

```

```

1 # Generate a random observed state
2 print("Observation state (randomly selected)",
3     small_environment.observation_space.sample())
4 # Generate a random action from the current state
5 print("Action (randomly selected):",
6     small_environment.action_space.sample())

```

```

1 # Generate an episode
2 observation, info = small_environment.reset() # state state_state
3 fig, ax = plt.subplots()
4 game_image = ax.imshow(small_environment.render())
5 MAX_STEPS = 100
6 refresh_rate = 1 # in (1 / seconds)
7 episode_over = False
8 n_steps = 0
9
10 while not episode_over and n_steps < MAX_STEPS:
11     n_steps += 1
12     action = small_environment.action_space.sample()
13     state, reward, terminated, truncated, info =
14         small_environment.step(action)
15     episode_over = terminated or truncated
16
17     ax.set_title(
18         'Step: {} State: {} Reward: {} Action:{}'.format(
19             n_steps,
20             state,
21             reward,
22             action_names[action],
23         )
24 )

```

```

25     display(fig)
26     time.sleep(1.0 / refresh_rate)
27     clear_output(wait=True) # Clear previous output
28     game_image.set_data(small_environment.render())
29
30 small_environment.close()

```

2.2. Step 1: Greedy and Epsilon greedy policies

Since Q-Learning is an off-policy algorithm, we have two policies. This means we're using a different policy for acting and updating the value function.

- Epsilon-greedy policy (acting policy)
- Greedy-policy (updating policy)

The greedy policy will also be the final policy we'll have when the Q-learning agent completes training. The greedy policy is used to select an action using the Q-table.

Epsilon-greedy is the training policy that handles the exploration/exploitation trade-off.

- With probability $1 - \epsilon$: we do exploitation (i.e. our agent selects the action with the highest state-action pair value).
- With probability ϵ : we do exploration (trying a random action).

As the training continues, we progressively reduce the epsilon value since we will need less and less exploration and more exploitation.

2.3. Step 2: Train the RL agent

TO DO: Implement the Q-learning algorithm in `reinforcement_learning.py`

Define the hyperparameters for the learning process. The exploration related hyperparameters are some of the most important ones.

- We need to make sure that our agent explores enough of the state space to learn a good value approximation. To do that, we need to have progressive decay of the epsilon.
- If you decrease epsilon too fast (too high `decay_rate`), you take the risk that your agent will be stuck in a local optimum, since your agent didn't explore enough of the state space and hence can't solve the problem.

```

1  # Training hyperparameters
2  n_training_episodes = 1000
3  max_steps = 100 # Maximum number of steps per episode
4  learning_rate = 0.7
5  gamma = 0.95 # Discount factor
6  # Exploration parameters
7  max_epsilon = 1.0 # Initial exploration probability
8  min_epsilon = 0.05 # Minimum exploration probability
9  decay_rate = 0.0005 # Exponential decay rate for the exploration
   probability
10
11 # Initialize Q-table
12 Qtable_small = np.zeros(
13     (
14         small_environment.observation_space.n,
15         small_environment.action_space.n
16     )
17 )
18 # Learn Q-table
19 Qtable_small = q_learning(
20     small_environment,
21     n_training_episodes,

```

```
22     max_steps ,
23     learning_rate ,
24     gamma ,
25     min_epsilon ,
26     max_epsilon ,
27     decay_rate ,
28     Qtable_small ,
29 )

1 frames = rl_utils.generate_greedy_episode(small_environment ,
    Qtable_small)
2 rl_utils.show_episode(frames , interval=250)
```

2.4. A more challenging problem

We're ready now to find our way in more challenging environments.

```
1 large_environment = gym.make(
2     'FrozenLake-v1',
3     map_name='8x8',
4     is_slippery=False,
5     render_mode='rgb_array'
6 )
7 n_states = large_environment.observation_space.n
8 print("There are ", n_states, " possible states")
9 n_actions = large_environment.action_space.n
10 print("There are ", n_actions, " possible actions")

1 n_training_episodes = 30000
2 max_steps = 500
3 learning_rate = 0.5
4 gamma = 0.99
5 max_epsilon = 1.0
6 min_epsilon = 0.2
7 decay_rate = 0.0001
8 # Initialize Q-table
9 Qtable_large = np.zeros(
10     (
11         large_environment.observation_space.n,
12         large_environment.action_space.n
13     )
14 )
15 # Learn Q-table
16 Qtable_large = q_learning(
17     large_environment ,
18     n_training_episodes ,
19     max_steps ,
20     learning_rate ,
21     gamma ,
22     min_epsilon ,
23     max_epsilon ,
24     decay_rate ,
25     Qtable_large ,
26 )
```

```

1 frames = rl_utils.generate_greedy_episode(large_environment,
      Qtable_large)
2 rl_utils.show_episode(frames, interval=250)

```

2.5. Slippery environment

Our environment is now slippery, meaning the agent sometimes slips and moves in an unintended direction.

```

1 slippery_environment = gym.make(
2     'FrozenLake-v1',
3     map_name='8x8',
4     is_slippery=True,
5     render_mode='rgb_array',
6 )

```

To visualize the challenges in this environment, let's make our agent go right several times and see what happens:

```

1 go_right = []
2 action = 2 # go-right
3 n_steps = 20
4 state, info = slippery_environment.reset()
5 for i in range(n_steps):
6     go_right.append(slippery_environment.render()) # Capture current
      frame
7     slippery_environment.step(action)
8 rl_utils.show_episode(go_right, interval=250)

```

Let's see what happens when we train our agent in the same way as before.

```

1 # TO DO: Training hyperparameters
2 n_training_episodes = 30000
3 max_steps = 700
4 learning_rate = 0.7
5 gamma = 0.99
6 max_epsilon = 1
7 min_epsilon = 0.1
8 decay_rate = 0.00001
9
10 Qtable_slippery = np.zeros((slippery_environment.observation_space.n,
      slippery_environment.action_space.n))
11 Qtable_slippery = q_learning(
12     slippery_environment,
13     n_training_episodes,
14     max_steps,
15     learning_rate,
16     gamma,
17     min_epsilon,
18     max_epsilon,
19     decay_rate,
20     Qtable_slippery,
21 )

```

Now we fix it:

```

1 # TO DO: Training hyperparameters

```

```
2 n_training_episodes = 100000
3 max_steps = 800
4 learning_rate = 0.1
5 gamma = 0.995
6 max_epsilon = 1
7 min_epsilon = 0.5
8 decay_rate = 0.00001
9
10 Qtable_slippery = np.zeros((slippery_environment.observation_space.n,
11                             slippery_environment.action_space.n))
12 Qtable_slippery = q_learning(
13     slippery_environment,
14     n_training_episodes,
15     max_steps,
16     learning_rate,
17     gamma,
18     min_epsilon,
19     max_epsilon,
20     decay_rate,
21     Qtable_slippery,
22 )
```

3. Exercise 3: Train the RL agent using SARSA

TO DO: Implement the SARSA learning algorithm in `reinforcement_learning.py`

```
1 slippery_environment = gym.make(
2     'FrozenLake-v1',
3     map_name='8x8',
4     is_slippery=True,
5     render_mode='rgb_array',
6 )
7 n_training_episodes = 100000
8 max_steps = 800
9 learning_rate = 0.1
10 gamma = 0.995
11 max_epsilon = 1.0
12 min_epsilon = 0.5
13 decay_rate = 0.0001
14 # Initialize Q-table
15 Qtable_slippery = np.zeros(
16     (
17         slippery_environment.observation_space.n,
18         slippery_environment.action_space.n
19     )
20 )
21 # Learn Q-table
22 Qtable_slippery = sarsa_learning(
23     slippery_environment,
24     n_training_episodes,
25     learning_rate,
26     gamma,
27     min_epsilon,
28     max_epsilon,
29     decay_rate,
```

```

30     max_steps ,
31     Qtable_slippery ,
32 )

```

4. Exercise 4: Compare SARSA and Q-learning

Answer these questions:

- What is "episode reward" in RL and how is it related to the performance of an RL agent?
- What is "episode length" in RL and how is it related to the performance of an RL agent?
- What is "training error" in RL and how is it related to the performance of an RL agent?
- Provide an explicit expression of the training error in the cases explored.

4.1. What is episode reward in RL and how is it related to the performance of an RL agent?

Answer: In Reinforcement Learning (RL), the episode reward is the total cumulative reward that an agent receives during a single episode — from the initial state to a terminal state.

$$R^{(i)} = \sum_{t=0}^{T_i} r_t \quad (1)$$

Where:

- T_i is the total number of time steps in episode i
- r_t is the reward received at time step t

A higher episode reward generally indicates better performance. For example, in environments like FrozenLake, an agent receives a reward of 1 for reaching the goal. So, an episode reward of 1 means success, while 0 means failure. Tracking the average episode reward over time helps us understand how well the agent is learning.

4.2. What is episode length in RL and how is it related to the performance of an RL agent?

Answer: The episode length is the number of steps taken before the episode ends.

$$\text{Episode Length} = T \quad (2)$$

Where T is the total number of steps in an episode. Episode length must be interpreted alongside the episode reward:

- Short + reward = 1 → efficient path to goal
- Long + reward = 1 → success but with a suboptimal path
- Short + reward = 0 → early failure (e.g., fell into a hole)
- Long + reward = 0 → wandering without reaching the goal

In general, shorter episodes with higher rewards indicate that the agent is learning an effective policy.

4.3. What is training error in RL and how is it related to the performance of an RL agent?

Answer: The training error in RL typically refers to the Temporal Difference (TD) error, which quantifies how much the Q-value estimate changes during learning.

In Q-learning, the TD error is:

$$\delta_t = r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \quad (3)$$

In SARSA, the TD error is:

$$\delta_t = r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \quad (4)$$

Where:

- r_t is the reward at time step t
 - γ is the discount factor
 - $Q(s, a)$ is the Q-value of state s and action a
- The average training error per episode can be calculated as:

$$\text{Training Error} = \frac{1}{T} \sum_{t=0}^T |\delta_t| \quad (5)$$

- A high training error indicates that the agent is still learning (i.e., Q-values are changing significantly).
- A low training error suggests that the agent's policy is stabilizing, which often correlates with improved and consistent performance.

5. Exercise 5: Compare SARSA and Q-learning plots

- Use matplotlib to compare the learning curves of SARSA and Q-learning, in terms of episode reward, episode length, and training error.
- Discuss the results obtained.

```
1 from rl_comparatives import *
2 from ex5 import plot1
3 plot1()
```

Q-learning learns faster: its average reward climbs by episode 200 and, despite a dip around 600, settles near 0.50 by episode 1000. SARSA takes longer to improve but rises more smoothly, eventually peaking just above 0.55. Both methods cut their episode lengths from dozens of random steps down to about 4–10 moves, though Q-learning stops producing very long runs a bit sooner.

In terms of TD error, Q-learning's estimates converge to near zero around episode 700, while SARSA's error stays around 0.1–0.2 as it keeps updating on-policy. In short, Q-learning is quicker but a bit wobbly; SARSA is steadier and edges out slightly higher final rewards.

```
1 from ex5 import plot2
2 plot2()
```

In the FrozenLake environment, Q-learning learned successfully: rewards increased, episode lengths dropped, and training error decreased over time. SARSA, however, failed to learn—the reward stayed at zero, episodes got longer, and the agent didn't improve. This likely happened because SARSA was too cautious and didn't explore enough.

6. Exercise 6: Train a deep Q-learning agent

```
1 from ex6 import *
2 agent = run_training()
```

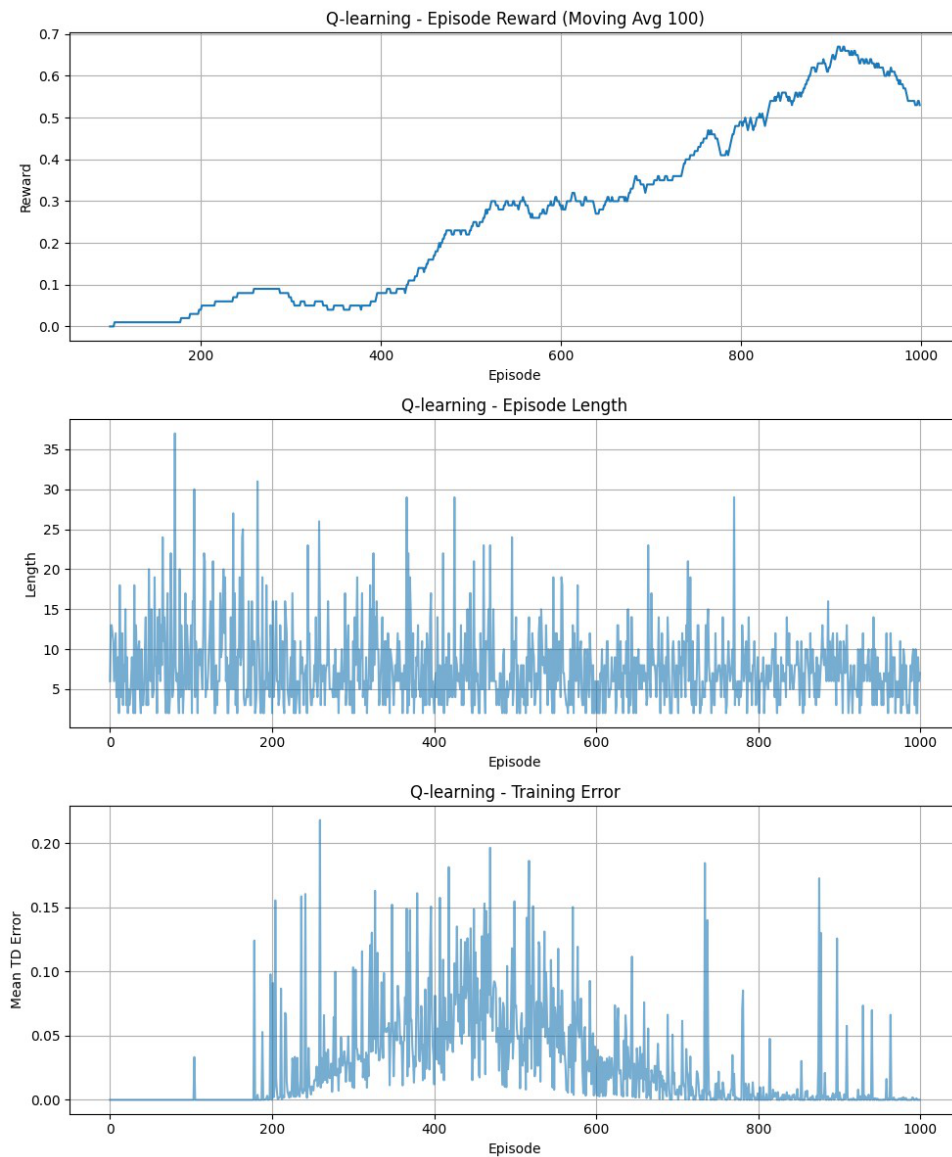


Figure 1. Plot 1 Results

Episode: 0/50, Score: 34.0, Epsilon: 1.00
 Episode: 10/50, Score: 19.0, Epsilon: 0.53
 Episode: 20/50, Score: 24.0, Epsilon: 0.25
 Episode: 30/50, Score: 17.0, Epsilon: 0.10
 Episode: 40/50, Score: 92.0, Epsilon: 0.01
 Test score: 122.0

```
1 rendering(agent) # a pygame window will open
```

6.1. Deep Q-Learning Agent

Deep Q-Learning (DQN) combines Q-learning with deep neural networks to handle complex state spaces. Instead of using a Q-table, it uses a neural network to approximate Q-values, enabling it to work with continuous or high-dimensional state spaces. In this implementation, we're training an agent to balance a pole on a moving cart by learning optimal actions through trial and error, using experiences stored in a replay buffer to improve stability and convergence.

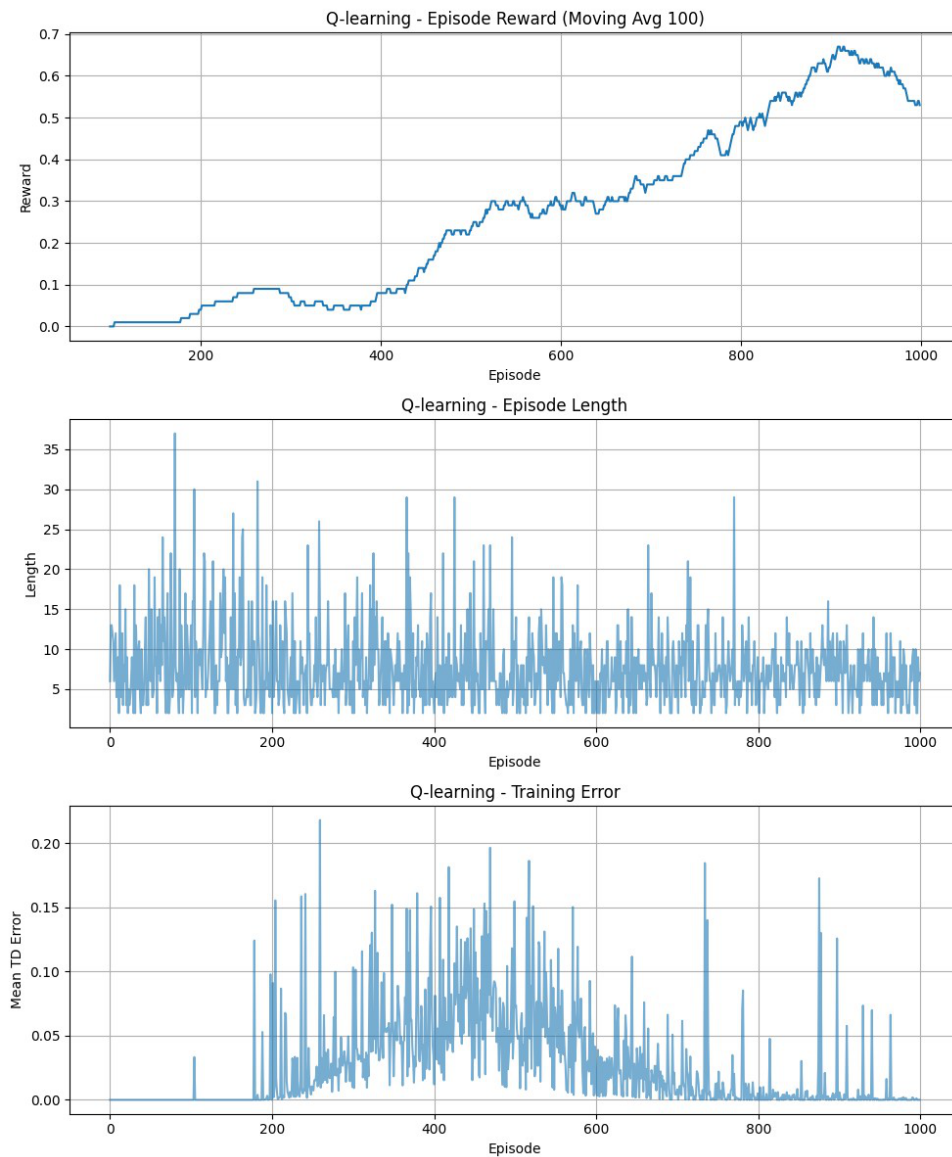


Figure 2. Plot 2 Results

7. References

- Suárez, A. (s.f.). Reinforcement Learning [Slides].
- Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press. <http://incompleteideas.net/book/RLbook2020.pdf>
- OpenAI. (2023). ChatGPT [Large language model]. <https://chat.openai.com/>
- DeepLizard. (2019, Jan 21). Q-learning vs SARSA | Reinforcement Learning [Video]. YouTube. https://www.youtube.com/watch?v=0g4j2k_Ggc4
- Hugging Face. (2022). Reinforcement Learning with Q-learning. <https://huggingface.co/learn/deep-rl-course/unit1/introduction>
- Towards Data Science. (2020). Learning with SARSA, a good alternative to Q learning algorithm <https://towardsdatascience.com/reinforcement-learning-with-sarsa-a-good-alternative->